

TP Programmation Orientée Objet

Langage Python

Avant-propos

Vous avez normalement reçu une invitation par mail pour pouvoir vous connecter au GitLab de l'université. Il faut la valider en cliquant sur « Join now », puis vous connecter en utilisant le LDAP de l'université et vos identifiants Sorbonne Université.

Une fois connecté au GitLab SU, vous pouvez aller sur le dépôt du TP qui est à l'adresse suivante :

<https://gitlab.sorbonne-universite.fr/poo-ei-2025/tp>

La première chose à faire est un *fork* du dépôt en cliquant sur le bouton correspondant. Sur la page qui suit vous devez sélectionner le *namespace* qui correspond à votre groupe de TP (tpa à tpe) puis cliquez sur « Fork project ». Il faudra nommer votre *fork* en y faisant apparaître votre nom.

Une fois le *fork* effectué, vous pouvez *cloner* le dépôt en cliquant sur le bouton « clone », en copiant l'*url* `https` et en faisant la commande `git clone url` dans votre répertoire de travail. Vous êtes encouragé à faire des *commits* régulier de votre travail, ceux-ci rentreront en compte dans l'évaluation.

Pour assurer que ce que vous codez est fonctionnel tout au long du TP, vous êtes encouragé à tester chaque classe/fonctionnalité. Pour cela il est recommandé d'utiliser `pytest`. Vous avez un exemple de son fonctionnement dans le répertoire `tp1/tests` du dépôt. Pour l'utiliser, il suffit de lancer la commande `pytest` depuis ce répertoire `tests`.

Il y a un `README` à la racine du projet qui explique tout cela.

Sujet

On souhaite modéliser un jeu de cartes faisant intervenir différents types de personnages. Ces personnages sont caractérisés par leur nom et un nombre de points de vie initial attribué aléatoirement. Ces personnages peuvent combattre et/ou soigner. Dans le cadre de ce TP les personnages se restreindront à :

- des guerriers pouvant combattre
- des soigneurs pouvant soigner
- et des paladins pouvant combattre et soigner.

Les paladins ne combattent ni ne soignent de la même façon que les guerriers ou les soigneurs. Les personnages pouvant combattre ont des points d'attaques qui déterminent le nombre de points de vie que perdra leur adversaire lorsqu'ils le blessent.

Les personnages pouvant soigner ont des points de soin qui déterminent le nombre de points de vie qu'ils redonnent à un personnage lorsqu'ils le soignent.

On associe également aux personnages un message permettant de décrire chaque action (combat et soin) qu'ils font.

Le module `random` propose pas mal de fonctionnalités autour des nombres aléatoires. Par exemple la fonction `randint(a, b)` retourne un entier aléatoire compris entre `a` et `b`.

Question 1

Modéliser les différents personnages, en veillant à avoir plusieurs personnages de chaque type (par exemple plusieurs guerriers avec des caractéristiques/attaques différentes). Cette modélisation devra utiliser des classes abstraites et interfaces et avoir au moins 4 niveaux de hiérarchie.

Coder en Python cette modélisation. Tester vos classes avec `pytest`.

Question 2

Redéfinissez la méthode `__str__` qui retourne une chaîne de caractères qui servira à l’affichage des caractéristiques des personnages. Vous pourrez utiliser les méthodes d’introspection qu’offre Python (`type`, `isinstance`, `issubclasse`, etc.). Tester votre code avec `pytest`.

Question 3

Implémenter des guerriers, soigneurs et paladins qui combattront et soigneront différemment les uns des autres. Voici quelques exemples :

- l’attaquant ne blesse son adversaire que si celui-ci est proche : à chaque attaque on pourra déterminer aléatoirement la distance entre les personnages
- certains combattant font plus de dommages lorsqu’ils sont loin
- certains combattants peuvent subir eux-mêmes des dommages lorsqu’ils attaquent
- le soigneur ne peut soigner que si le blessé est proche
- le soigneur perd des points de vie lorsqu’il soigne
- la récupération du blessé prend plusieurs jours
- ...

Tester votre code avec `pytest`.

Question 4

Tester votre programme en utilisant le fichier `main_tp1.py` fournit dans le dépôt GitLab.

Vous devez arriver ici à la fin du premier TP

Question 5

Modifier votre programme pour que l’ensemble des personnages intervenant dans votre jeu soit dynamique en utilisant la collection appropriée. Tester votre code avec `pytest`.

Question 6

Trier l’ensemble des personnages selon différents critères :

- le nombre de points de vie restant,
- le nombre de points d’attaque ou de soin restant,
- le type de personnage (soigneur ou guerrier) et les points correspondants.

Tester votre code avec `pytest`.

Question 7

Rajouter à votre code de quoi permettre de rajouter à la volée lors de la création des personnages leur action en utilisant par exemple les fonctions `lambda`. Tester votre code avec `pytest`.

Question 8

Ajouter ce qu’il faut pour traiter les exceptions. Tester votre code avec `pytest`.

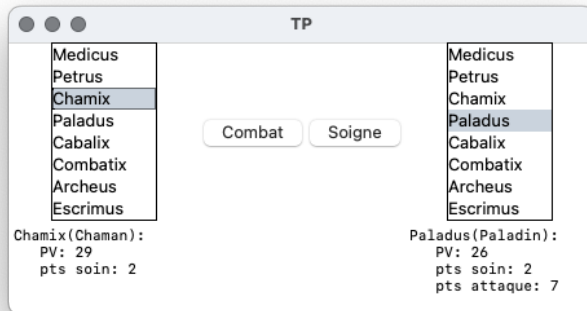
Question 9

Tester votre programme en utilisant le fichier `main_tp2.py` fournit dans le dépôt GitLab.

Vous devez arriver ici à la fin du deuxième TP

Question 10

Utiliser le module `tkinter` (ou <https://nicegui.io/>) pour écrire l'application permettant d'obtenir l'interface graphique suivante en utilisant les *widgets* `Listbox`, `Text`, `Button` et en les plaçant en utilisant le *layout* `grid` :



Question 11

Ecrire la méthode permettant d'afficher les caractéristiques du personnage sélectionné dans la `Listbox` dans le *widget* `Text` correspondant.

Question 12

Ecrire la méthode permettant de déclencher l'action entre les 2 personnages correspondants et de mettre à jour l'affichage des *widgets* `Text` correspondants.

Question 13

Connecter ces méthodes aux *widgets* correspondants à l'aide de la fonction `bind`.

Question 14

Utiliser la méthode `messagebox.showinfo` pour afficher les exceptions dans une fenêtre *popup*.